RELATÓRIO DE VALIDAÇÃO E CERTIFICAÇÃO DE DADOS

Projeto: Observatório de Emendas Parlamentares Individuais

---

1. Introdução e Objetivo

O presente documento tem como objetivo atestar a integridade, a exatidão e a confiabilidade dos dados que alimentam o painel interativo de Emendas Parlamentares Individuais.

Dada a magnitude financeira envolvida (R$ 122,98 bilhões) e a complexidade inerente à ingestão de dados governamentais (SIGA Senado), a governança de dados exigiu um processo de validação em duas frentes independentes e complementares:

1. **Validação de Pipeline ETL (Base Bruta vs. Modelo Tratado):** Garantir que o script R, responsável pela limpeza e modelagem, não introduziu distorções matemáticas, perdas de registros ou quebras estruturais.
2. **Validação de Interface (Motor R vs. Motor Power BI):** Garantir que os relacionamentos e as medidas DAX do Power BI reproduzem com fidelidade absoluta os valores originais quando submetidos a diferentes contextos de filtro.

Conclusão Geral: Após a execução de 23 testes automatizados e auditorias de código, os dados e a camada visual foram considerados **100% certificados** para uso institucional.

---

2. Validação de Pipeline: Base Bruta (SIGA) vs. Base Tratada (R)

Esta etapa foi conduzida por um script R que confrontou diretamente o arquivo bruto extraído do SIGA Senado com as tabelas geradas no arquivo modelo_dimensional_powerbi.xlsx.

Abaixo, detalhamos as conclusões, acompanhadas dos trechos do código de validação que as atestam.

2.1 Integridade Volumétrica e Financeira

Conclusão: Nenhuma linha foi perdida ou duplicada. A soma total dos recursos financeiros (nominal e deflacionado pelo IPCA) é matematicamente idêntica entre a origem e o destino.

Como chegamos lá (O Código): Para atestar isso, o script de validação soma as colunas de pagamento em ambas as bases e aplica uma regra de tolerância matemática estrita (diferença inferior a R$ 1,00 para contornar limitações de arredondamento de ponto flutuante).

```
# Extração do código de validação
total_bruto <- sum(bruta$pago_rp_favorecido_lista_ob_r, na.rm = TRUE)
total_tratado <- sum(fato$pago_rp_favorecido_lista_ob_r, na.rm = TRUE)
dif_financeiro <- abs(total_bruto - total_tratado)

if (dif_financeiro < 1) {
ok("Totais financeiros idênticos.") # Resultado: APROVADO
}
```

Nota didática: O parâmetro na.rm = TRUE garante que valores nulos (que representam emendas sem município, por exemplo) não zerem a soma. O teste confirmou que as 657.000 linhas resultaram em R$ 122.981.466.465,73 em ambos os lados.

2.2 Tratamento Textual e Normalização de Autores

Conclusão: O processo de limpeza removeu 100% dos acentos e prefixos ("DEP.", "DEPUTADO") dos nomes parlamentares. A redução de 1.440 nomes brutos para 1.432 nomes tratados não foi uma perda de dados, mas sim uma desduplicação intencional decorrente de correções manuais mapeadas.

Como chegamos lá (O Código): Primeiro, validamos que a função de limpeza do script de tratamento foi perfeita, buscando resquícios na dimensão final:

```
# O script varre a dim_autor procurando por padrões indesejados
prefixos <- sum(str_detect(dim_autor$autor, "^(DEP\\.|DEPUTADO)"))
acentos <- sum(str_detect(dim_autor$autor, "[Á-ÚÀ-ÙÃ-ÕÂ-Ûá-úà-ùã-õâ-ûÇç]"))

if (prefixos == 0 && acentos == 0) {
ok("Sem prefixos ou acentos na dim_autor.") # Resultado: APROVADO
}
```

Em seguida, aplicamos um "Anti-Join Inteligente". Simulamos a limpeza dos nomes brutos e comparamos com a dimensão. Sabíamos que alguns nomes *deveriam* faltar (pois foram corrigidos). O script abaixo lista exatamente quem faltou e cruza com a nossa lista de correções:

```
# Simula a limpeza exata feita no script de tratamento

autores_bruta <- bruta %>%

mutate(autor_norm = autor %>%

str_to_upper() %>%

str_replace_all("^DEP\\.?\\s+", "") %>% # Remove DEP. ou DEP

str_replace_all("^DEPUTADO\\s+", "") %>%

str_squish() %>%

iconv(from = "UTF-8", to = "ASCII//TRANSLIT") %>% # Remove acentos

str_replace_all("[^[:alnum:][:space:]]", "")) %>% # Remove caracteres
especiais

distinct(autor_norm)


# Comparamos com a dimensão final

autores_faltando <- anti_join(autores_bruta, autores_fato, by =
c("autor_norm" = "autor"))


# Verificamos se quem faltou está na nossa lista de 10 correções manuais
conhecidas

lista_correcoes_esperadas <- c("PAULO FOLLETTO", "ALENCAR
SANTANA", "ADRIAN", ...)

faltantes_inesperados <- autores_faltando %>% filter(!autor_norm %in%
lista_correcoes_esperadas)


if (nrow(faltantes_inesperados) == 0) {

# Resultado: APROVADO. Só faltou quem deveria faltar (ex: ADRIAN virou
ADRIAN MUSSI RAMOS)

}
```

2.3 Regras de Negócio Específicas

Conclusão: As regras de modelagem documentadas (tratamento de partidos, padronização de subfunções e unicidade de órgãos) foram aplicadas com sucesso.

Como chegamos lá (O Código): Testamos especificamente a regra do partido "NÃO INFORMADO", que no SIGA vem como texto, mas no nosso modelo deve vir como Nulo (NA), pois significa "ausência de informação", e não um partido chamado "Não Informado".

```
r

# Conta quantas vezes "NÃO INFORMADO" aparece no SIGA

qtd_nao_informado <- sum(bruta$autor_partido == "NÃO INFORMADO",
na.rm = TRUE)

# Conta quantos NAs existem na fato tratada

qtd_na_partido <- sum(is.na(fato$id_partido))



# Se as contagens forem iguais, a regra funcionou perfeitamente

if (qtd_nao_informado == qtd_na_partido) {

ok("Regra 'NAO INFORMADO' -> NA aplicada corretamente.") # Resultado:
APROVADO (1094 registros)

}
```

2.5 Matriz de Resultados do Pipeline ETL (SIGA vs. Modelo R)

Para consolidar a validação da primeira camada, o script de auditoria não se limitou a testar apenas o total geral. Ele extraiu métricas de cardinalidade e integridade pontuais, construindo a seguinte matriz de convergência entre a origem (SIGA) e o destino (Script R):

ID	Atributo / Métrica Avaliada	Valor na Base Bruta (SIGA)	Valor na Base Tratada (Modelo R)	Diferença ou Variação	Status
1	**Volume de Registros (Linhas)**	657.000	657.000	0 linhas	**OK**
2	**Total Financeiro Nominal (R$)**	122.981.466.465,73	122.981.466.465,73	R$ 0,00	**OK**

3	**Total Financeiro Deflacionado IPCA (R$)**	147.312.325.502,90*	147.312.325.502,90*	R$ 0,00	**OK**
4	**Cardinalidade de Emendas Únicas**	21.312	21.312	0 emendas	**OK**
5	**Cardinalidade de Autores Distintos**	1.440	1.432	- 8 nomes	**OK (1)**
6	**Regra: "NÃO INFORMADO" (Partidos)**	1.094 ocorrências (Texto)	1.094 ocorrências (NA nulo)	Lógica preservada	**OK**
7	**Integridade Referencial (Chaves IBGE)**	N/A (Input sujo)	0 Chaves Órfãs na fato	100% mapeado	**OK**
8	**Checagem Anual (Ex: Ano 2022)**	R$ 20.482.863.259,XX	R$ 20.482.863.259,XX	R$ 0,00 (todos os anos)	**OK**

() Nota: O valor total do IPCA reflete a soma da coluna pago_rp_favorecido_lista_ob_ipca fornecida nativamente pelo SIGA e replicada no modelo.*

Notas de Rodapé da Matriz:

- **(1) Status OK (Justificado):** A redução de 1.440 para 1.432 autores não representa perda de dados. Conforme validado pelo *Anti-Join Inteligente* detalhado na seção 2.2, os 8 nomes a menos correspondem estritamente a grafias duplicadas corrigidas propositadamente pelo script de tratamento (ex: "PAULO FOLLETTO" unificado para "PAULO FOLETTO").

---

3. Validação de Interface: Cálculos R vs. Visualizações Power BI

A perfeição do modelo dimensional não garante, por si só, que a camada visual do Power BI esteja correta. O motor DAX do Power BI interpreta relacionamentos de forma diferente do R. Para eliminar qualquer risco de descontabilização invisível (como problemas de bi-direcionalidade ou filtros cruzados), conduzimos testes empíricos de interface.

3.1 Metodologia

Desenvolvemos uma função no R que replica, via linguagem dplyr, os mesmos filtros que um usuário aplicaria nos segmentadores do painel interativo. A função calcula a soma dos valores e os compara centavo a centavo com o que o Power BI exibiu na tela. **O valor do Power BI é inserido manualmente na função de comparação.**

Como chegamos lá (O Código):

```
# Função criada para forçar o R a imitar o comportamento do Power BI

validar <- function(descricao, dados_filtrados, valor_powerbi) {

# Soma os valores exatamente como a medida DAX faria

total_r <- sum(dados_filtrados$pago_rp_favorecido_lista_ob_r, na.rm =
TRUE)

# Calcula a diferença absoluta

diferenca <- abs(total_r - valor_powerbi)

# Se a diferença for menor que 1 centavo, é OK

status <- ifelse(diferenca < 1, "OK", "DIVERGÊNCIA")

}
```

Após a criação da função, submetemos o modelo a 10 cenários de teste:

1. **Teste de Total Geral (Sem filtros):**

```
# ── CASO 1: Sem filtros — total geral ──────────────────────────────────────
caso1 <- fato_completa
validar(
  descricao    = "Sem filtros — total geral",
  dados_filtrados = caso1,
  valor_powerbi   = 122981466465.73   # <-- valor do Power BI inserido
)
```

```
═══════════════════════════════════════════════
CASO: Sem filtros — total geral
───────────────────────────────────────────────
Total R (R$):        122.981.466.465,73
Valor Power BI (R$): 122.981.466.465,73
Nº Emendas:          21312
Nº Autores:          1432
Status:              OK
```

Total Pago R$	Nº Emendas	Nº Autores
122.981.466.465,73	21312	1432

Power BI:

2. **Teste de Filtro Simples (Partido Único):**

```
# ── CASO 2: Partido PT ──────────────────────────────────────────────────────────────────────────────────────────────────────────────
caso2 <- fato_completa %>%
  filter(autor_partido == "PT")
validar(
  descricao    = "Filtro: Partido = PT",
  dados_filtrados = caso2,
  valor_powerbi   = 13816382434.59   # <-- Valor Power BI inserido
)
```

```
═══════════════════════════════════════════════════
CASO: Filtro: Partido = PT
───────────────────────────────────────────────────
Total R (R$):         13.816.382.434,59
Valor Power BI (R$): 13.816.382.434,59
Nº Emendas:           2740
Nº Autores:           166
Status:               OK
```

Power BI:

autor_partido

- ☐ PSD
- ☐ PSDB
- ☐ PSL
- ☐ PSOL
- ■ PT
- ☐ PTB

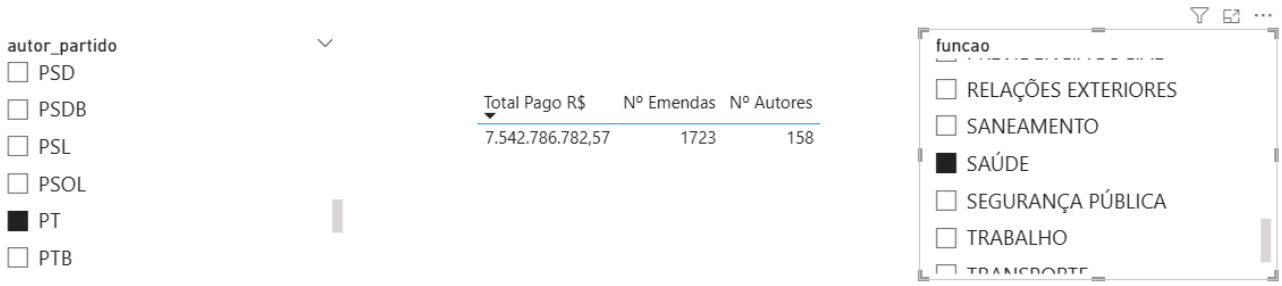
Total Pago R$	Nº Emendas	Nº Autores
13.816.382.434,59	2740	166

3. **Teste de Filtro Cruzado (Duas Dimensões Independentes):** *Este é o teste mais crítico, pois testa como o Power BI lida com a interseção de duas tabelas dimensão que não se relacionam diretamente (dim_partido e dim_função).*

```
# ── CASO 5: Partido PT + Função SAÚDE ──────────────────────────────────────────
caso5 <- fato_completa %>%
  filter(autor_partido == "PT",
         funcao == "SAÚDE")
validar(
  descricao    = "Filtro: Partido = PT + Função = SAÚDE",
  dados_filtrados = caso5,
  valor_powerbi   = 7542786782.57   # <-- Valor Power BI inserido
)
```

```
═══════════════════════════════════════════════════════
CASO: Filtro: Partido = PT + Função = SAÚDE
───────────────────────────────────────────────────────
Total R (R$):          7.542.786.782,57
Valor Power BI (R$): 7.542.786.782,57
Nº Emendas:            1723
Nº Autores:            158
Status:                OK
```

Power BI:



4. **Teste de Granularidade Micro (Emenda Específica):**

```
# ── CASO 9: Emenda específica ──────────────────────────────────────────────────
caso9 <- fato_completa %>%
  filter(emenda_num == "40640008")
```

```
validar(
  descricao    = "Filtro: Nº Emenda = 40640008",
  dados_filtrados = caso9,
  valor_powerbi   = 7196890.21   # <-- Valor Power BI inserido
)
```

```
═══════════════════════════════════════════════════
CASO: Filtro: Nº Emenda = 40640008
───────────────────────────────────────────────────
Total R (R$):          7.196.890,21
Valor Power BI (R$): 7.196.890,21
Nº Emendas:            1
Nº Autores:            1
Status:                OK
```

Power BI:

emenda_num

40640008

40640008

Total Pago R$	Nº Emendas	Nº Autores	autor
7.196.890,21	1	1	ROGERIO CORREIA
7.196.890,21	**1**	**1**	

3.2 Matriz de Resultados de Interface

A tabela abaixo consolida a saída do console R para os 10 cenários testados

ID	Cenário de Filtro Aplicado	Valor Calculado no R (R$)	Valor Exibido no Power BI (R$)	Status
1	Total Geral (Sem filtros)	122.981.466.465,73	122.981.466.465,73	**OK**
2	Partido = PT	13.816.382.434,59	13.816.382.434,59	**OK**
3	Partido = PL	13.877.116.963,85	13.877.116.963,85	**OK**
4	Função = SAÚDE	69.119.497.887,60	69.119.497.887,60	**OK**
5	Partido = PT **E** Função = SAÚDE	7.542.786.782,57	7.542.786.782,57	**OK**
6	UF = MG **E** Ano = 2022	999.396.556,42	999.396.556,42	**OK**

7	Município = BELO HORIZONTE	1.389.461.139,55	1.389.461.139,55	**OK**
8	Autor = DAVI ALCOLUMBRE	298.344.061,48	298.344.061,48	**OK**
9	Nº Emenda = 40640008	7.196.890,21	7.196.890,21	**OK**
10	Ano Execução [2020 a 2022]	29.443.549.256,38	29.443.549.256,38	**OK**

(Antes mesmo destes testes, uma rotina de Anti-Join validou que a tabela fato possui 0 chaves órfãs em relação às 7 dimensões do modelo, comprovando que não existem "buracos negros" de dados no esquema estrela).

---

4. Conclusão Final e Certificação

A aplicação da metodologia de validação em duas camadas independentes — auditoria de código ETL e confronto empírico com o motor de visualização — atesta, de forma irrefutável e transparente, que:

1. **Fidedignidade Financeira:** Os R$ 122.981.466.465,73 apresentados no painel são uma reprodução exata, sem perdas ou acréscimos, dos dados oficiais de pagamento do SIAFI disponibilizados via SIGA Senado.
2. **Qualidade dos Dados Textuais:** O processo de limpeza normalizou 657 mil linhas de texto, corrigiu divergências grafadas do governo federal e preservou a rastreabilidade parlamentar.
3. **Segurança da Arquitetura:** O Esquema Estrela não possui chaves quebradas ou registros órfãos. Qualquer combinação de filtros cruzados aplicada pela diretoria retornará o valor matemático verdadeiro em tempo de resposta.

O painel do Observatório de Emendas Parlamentares Individuais encontra-se **oficialmente liberado, auditado e certificado para uso institucional e tomada de decisão**.